Software Requirements Specification

Paper Generator Web Application

Prepared By:

Talha and Sameer

BS Information Technology

Government Islamia Graduate College, Kasur

Instructor: Faisal Hameed

February 28, 2026

Contents

1 Introduction **4**

1.1 Purpose 4

1.2 Scope 4

1.3 Overview 5

1.4 Definitions, Acronyms, and Abbreviations 6

2 Overall Description **7**

2.1 Product Perspective 7

2.2 Product Functions 7

2.3 User Classes and Characteristics 8

2.4 Operating Environment 8

2.5 Design and Implementation Constraints 9

2.6 Assumptions and Dependencies 9

3 System Features **10**

3.1 Functional Requirements 10

3.1.1 User Authentication 10

3.1.2 Course Material Management 10

3.1.3 Question Bank Management 10

3.1.4 Paper Generation 10

3.1.5 Export and Printing 11

3.1.6 Notifications and Feedback 11

3.1.7 Admin Functions 11

3.2 Non-Functional Requirements 11

3.2.1 Performance 11

3.2.2 Usability 12

3.2.3 Reliability 12

3.2.4 Scalability 12

3.2.5 Security 13

3.2.6 Portability 13
3.2.7 Maintainability 14

4 Technology Stack **14**
4.1 Front End 14
4.2 Back End 14
4.3 Database 15
4.4 Development Tools 15

5 System Models **16**
5.1 Use Case Diagram 16
5.2 Use Case Diagram: Paper Generation 17
5.3 Use Case Diagram: Question Bank Management 19
5.4 Use Case Diagram: Course Material Management 21
5.5 Sequence Diagram: Paper Generation 23
5.6 Sequence Diagram: Question Bank Management 24
5.7 Sequence Diagram: Course Material Management 25
5.8 System State Diagram: Paper Generator Application 26
5.9 State Diagram: Paper Generation Process 26
5.10 State Diagram: Course Material Management 26
5.11 Class Diagram: Paper Generator Web Application 27
5.12 Class Diagram: Question Bank Management 28
5.13 Class Diagram: Export Module 29
5.14 System Activity Diagram: Paper Generator Application 31
5.15 Activity Diagram: Paper Generation 33
5.16 ER Diagram 34
5.17 Context Diagram 36
5.18 Gantt Chart 36

6 Test case: **37**
6.1 Test Case 1: Generate Paper with Valid Inputs 37

6.2 Test Case 2: Generate Paper with Insufficient Questions 38
6.3 Test Case 3: Manage Question Bank 39

1 Introduction

1.1 Purpose

In most academic institutions, teachers spend significant time manually preparing examination materials such as quizzes, assignments, midterm exams, and final exams. This process involves gathering questions from multiple sources, ensuring appropriate coverage of topics, and adjusting difficulty levels. Such manual effort is not only time-consuming but also increases the risk of errors, inconsistency, and repetition in question selection.

The Paper Generator Web Application is designed to address these challenges by automating the creation of academic assessments. The system provides a centralized question bank where teachers can store, categorize, and retrieve questions based on subject, topic, and difficulty level. It enables the generation of various types of assessments—including quizzes, assignments, midterm exams, and final exams—either randomly or through customized selection by the teacher.

A key feature of this system is its ability to generate papers according to difficulty levels, such as easy, moderate, or advanced, allowing teachers to tailor exams to the required standard of evaluation. This ensures balanced assessment while promoting fairness and efficiency.

The primary benefit of the Paper Generator Web Application is to enhance productivity, accuracy, and reliability in the academic evaluation process. By reducing the manual workload of teachers, the system not only saves time but also improves the quality and consistency of assessments. In the long run, it provides an adaptable solution that supports the needs of academic institutions at multiple levels of testing.

1.2 Scope

The Paper Generator Web Application is a web-based system designed to assist academic institutions in the creation of assessment materials, including quizzes, assignments, midterm examinations, final examinations, and other classroom tests. The system offers a centralized question bank where teachers can create, categorize, and retrieve questions

based on subject, topic, and difficulty level.

The application also enables teachers to upload course materials—such as PDFs, Word documents, and digital notes—which the system analyzes to automatically generate relevant questions. This ensures that the assessments produced are accurately aligned with the specific course content.

Teachers can generate examination papers either randomly or through manual selection of questions. The system supports difficulty-based paper generation (easy, moderate, advanced), ensures balanced distribution of questions, minimizes repetition, and allows exporting the final paper in printable formats such as PDF or Word.

This project is limited to the generation and management of assessment papers. It does not include automated grading, plagiarism detection, student management, result processing, or online examination delivery. Its primary objective is to reduce manual effort, enhance accuracy, and improve the efficiency of examination preparation for teachers and academic administrators.

1.3 Overview

The Paper Generator Web Application is a web-based system designed to automate the creation of academic assessments. It provides an efficient and reliable alternative to the traditional manual method of preparing quizzes, assignments, midterm exams, and final examinations. The system is intended primarily for teachers and academic staff involved in paper preparation across various educational levels.

The application allows users to upload course materials—such as PDFs, textbooks, and digital notes—from which the system can automatically generate relevant questions. A centralized question bank is also provided for the manual creation, categorization, and storage of questions.

The system supports the generation of examination papers based on difficulty levels (easy, moderate, advanced) and offers export options in both PDF and Word formats to facilitate distribution and printing. As a web-based platform backed by a secure database, the system ensures scalability, accessibility, and long-term maintainability for academic

institutions.

1.4 Definitions, Acronyms, and Abbreviations

Admin The system administrator who manages user accounts, maintains the question bank, and oversees overall system operations.

Automatic Paper Generator The proposed web application that automates the creation of quizzes, assignments, midterm, and final examination papers.

Authentication The process of verifying the identity of a user before granting access to the system.

Database A structured collection of data used to store and retrieve questions, user information, and exam templates.

Exam Template A predefined structure or format that specifies the distribution of questions in an examination paper (e.g., number of MCQs, short questions, long questions).

PDF (Portable Document Format) A digital file format used for exporting and distributing generated exam papers.

Question Bank A repository within the system where questions are stored, categorized by subject, topic, difficulty level, and type.

SRS (Software Requirements Specification) The document that specifies the functional and non-functional requirements of the system.

UI (User Interface) The part of the application that allows interaction between the user and the system.

2 Overall Description

2.1 Product Perspective

The Paper Generator Web Application is a web-based standalone system that automates the creation of quizzes, assignments, midterms, and final exams, replacing the manual paper preparation process.

Teachers can upload course materials (PDFs, Word documents, textbooks) or use a centralized question bank to generate questions automatically, organized by subject, topic, and difficulty.

The system allows paper generation by difficulty level — Easy, Moderate, or Advanced — and supports exporting papers in PDF or Word format for easy printing or digital use.

It runs as a web application accessible through modern browsers and uses a secure database, making it fully standalone, scalable, and reliable for academic institutions.

2.2 Product Functions

- **Course Material Management:** The system allows teachers to upload course materials such as PDFs, Word documents, and digital textbooks. It automatically analyzes the uploaded files and extracts relevant questions, which are then stored in the centralized question bankk.

- **Question Bank Management:**Teachers can manually create, edit, categorize, and store questions in the question bank. Questions can be organized and retrieved based on subject, topic, question type, and difficulty level to ensure structured content management.

- **Paper Generation:** The system can generate quizzes, assignments, midterm exams, and final examinations using either uploaded course material or existing question bank entries. Teachers can select the desired difficulty level (easy, moderate, or advanced), and the system ensures balanced distribution and avoids repetition of questions.

- **Export and Printing:** Generated papers can be exported in PDF or Word format. Before finalizing, teachers can preview the paper and make necessary adjustments, such as editing questions, rearranging order, or modifying marks distribution.

- **User Management (Optional):** The system may include user account creation and authentication for teachers. This allows secure access to uploaded materials, stored questions, and previously generated papers.

2.3 User Classes and Characteristics

- **Teachers / Professors:** Primary users responsible for uploading course material, managing the question bank, and generating assessments. They are expected to have basic computer skills and familiarity with web applications.

- **Administrators (Optional):** Responsible for managing user accounts and maintaining the system. They ensure proper access control and data integrity.

- **System Access Requirements:** All users require a modern web browser and internet connection. Teachers need access to course materials in standard formats (PDF, Word).

2.4 Operating Environment

The Paper Generator Web Application is a web-based system accessible through modern browsers such as Chrome, Edge, and Firefox. It runs on standard desktop or laptop computers and does not require specialized hardware.

The system uses a secure server-side database to store course materials, questions, and generated papers. Internet connectivity is required for full functionality, including uploading materials and exporting papers.

The application is designed to be scalable and platform-independent, allowing easy deployment in academic institutions of various sizes. It supports standard file formats (PDF, Word) for input and output.

2.5 Design and Implementation Constraints

- **Web-Based Platform:** The system must run as a web application compatible with modern browsers. It cannot rely on desktop-only software.
- **File Format Support:** Uploaded course materials must be in standard formats (PDF, Word) to ensure proper question extraction.
- **Internet Requirement:** A stable internet connection is necessary for uploading materials, generating papers, and exporting files.
- **Database Dependency:** The system relies on a secure server-side database for storing course materials, questions, and generated papers.
- **Scalability Limitations:** Performance may depend on server capacity and the number of concurrent users, especially in large institutions.

2.6 Assumptions and Dependencies

- **Teacher-Provided Materials:** The system assumes that teachers provide course materials in standard formats (PDF, Word) for question extraction. Improperly formatted files may affect functionality.
- **Internet Connectivity:** A stable internet connection is required for uploading materials, generating papers, and exporting files.
- **Browser Compatibility:** Users are expected to access the system through modern web browsers such as Chrome, Edge, or Firefox.
- **Server and Database Availability:** The system depends on a secure, functioning server and database to store course materials, questions, and generated papers. Any server downtime may affect operations.
- **User Competency:** Teachers using the system are assumed to have basic computer skills and familiarity with web applications.

3 System Features

3.1 Functional Requirements

3.1.1 User Authentication

- Teachers must register with name, email, and password.
- Teachers log in using valid credentials.
- The system supports password reset through email verification.

3.1.2 Course Material Management

- Teachers can upload course materials in PDF or Word format.
- Uploaded files are processed to extract questions automatically.
- The system validates file types before upload.

3.1.3 Question Bank Management

- Teachers can add, edit, delete, and categorize questions.
- Questions are organized by subject, topic, and difficulty.
- The system prevents duplication in the database.
- Search and filter options are available by keyword, topic, or difficulty.

3.1.4 Paper Generation

- Teachers can create quizzes, assignments, midterms, and finals.
- Papers can be generated randomly or through manual selection.
- Teachers can set difficulty levels (Easy, Moderate, Hard).
- Generated papers include metadata such as subject, exam type, and total marks.

3.1.5 Export and Printing

- Generated papers can be exported in PDF or Word format.
- Teachers can preview and edit papers before finalizing.
- The system supports printing and downloading exam papers.

3.1.6 Notifications and Feedback

- System provides success/error notifications for uploads, edits, and paper generation.
- Teachers receive confirmation when an exam paper is generated successfully.

3.1.7 Admin Functions

- Admins can manage teacher accounts.
- Admins can monitor system usage and database growth.
- Admins have access to audit logs for troubleshooting.

3.2 Non-Functional Requirements

3.2.1 Performance

- The system must generate exam papers within 3–5 seconds, even when processing large question banks.
- Response time for user interactions (e.g., searching questions, filtering topics) should not exceed 2 seconds.
- The application should be optimized to handle up to 10,000+ stored questions without significant delay.
- Performance testing must be conducted to ensure acceptable speed under normal and peak loads.

3.2.2 Usability

- The user interface must be simple, consistent, and intuitive, allowing teachers to navigate without prior technical knowledge.
- Common tasks (e.g., uploading materials, generating papers) should be achievable in no more than 3–4 clicks.
- The system should provide tooltips, error messages, and help guides to assist users when needed.
- Usability testing with representative users (teachers) must be performed to ensure ease of use.

3.2.3 Reliability

- The system must generate exam papers accurately without duplication or missing content.
- It should ensure proper and secure storage of course materials and the question bank.
- In case of failures such as network interruptions or system crashes, the system must recover without data loss.
- Regular backups and error-handling mechanisms should be in place to guarantee dependable operation.

3.2.4 Scalability

- The system should support multiple concurrent users without performance degradation.
- It must efficiently handle an expanding question bank as the database grows over time.

- The architecture should allow easy scaling of resources, such as storage and processing power, to meet future demands.
- The system should maintain consistent response times as the number of users and stored questions increases.

3.2.5 Security

- The system must protect course materials, question banks, and generated papers from unauthorized access.
- User authentication should be required before accessing the system.
- Role-based access control must be implemented to differentiate permissions between admins and teachers.
- All sensitive data should be encrypted during storage and transmission.
- Regular security audits and updates must be performed to safeguard against vulnerabilities.

3.2.6 Portability

- The system should be compatible with all major modern web browsers, including Chrome, Firefox, Edge, and Safari.
- It must run smoothly on different operating systems such as Windows, macOS, and Linux.
- The application should be accessible on various devices, including desktops, laptops, tablets, and smartphones.
- The design should follow responsive web standards to ensure a consistent user experience across platforms.

3.2.7 Maintainability

- Maintenance is the process of keeping the system in proper working condition after deployment.
- It includes fixing bugs, applying updates, and improving features to ensure smooth operation.
- Regular maintenance prevents major failures and extends the system's usable life.
- Maintenance activities include corrective, adaptive, perfective, and preventive actions.
- Effective maintenance ensures the system continues to meet user needs and adapts to new requirements.

4 Technology Stack

4.1 Front End

- The front end will be developed using **React.js** to build a dynamic and responsive user interface.
- **HTML5** and **CSS3** will be used for structuring and styling the application.
- **JavaScript (ES6+)** features will enhance interactivity and improve performance.

4.2 Back End

- The back end will be powered by **Node.js** for handling server-side logic efficiently.
- **Express.js** will be used as the framework to manage APIs and streamline backend development.
- **RESTful APIs** will connect the client-side with the server-side securely.

4.3 Database

- **MongoDB** will be used as the NoSQL database to store questions, templates, and user information.
- Its document-oriented model supports scalability and flexibility for the growing question bank.
- Indexing and replication will ensure efficient data retrieval and reliability.

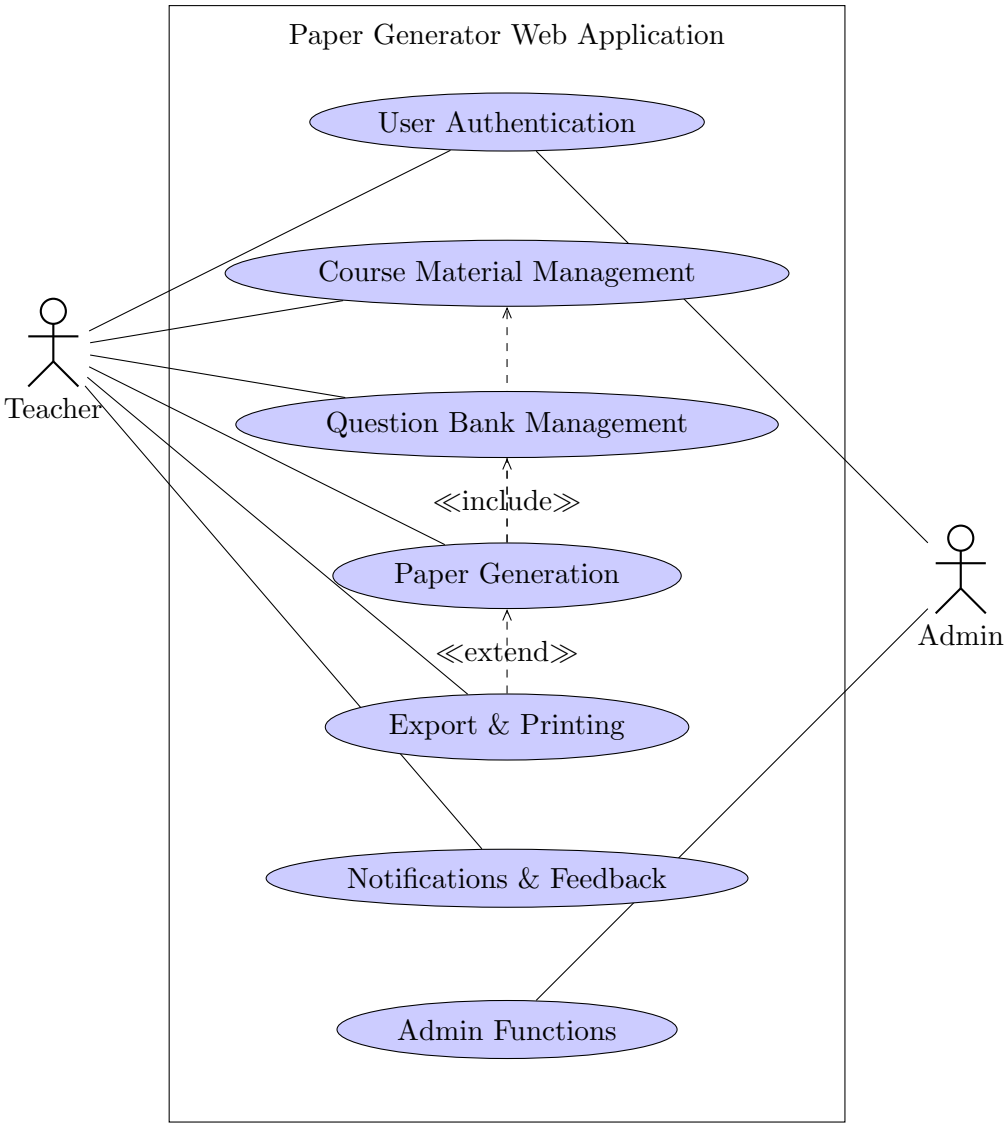
4.4 Development Tools

- **Visual Studio Code (VS Code)** will serve as the primary IDE for development.
- **Git and GitHub** will be used for version control and collaborative development.
- **Postman** will assist in testing APIs, while browser developer tools will support debugging and optimization.

5 System Models

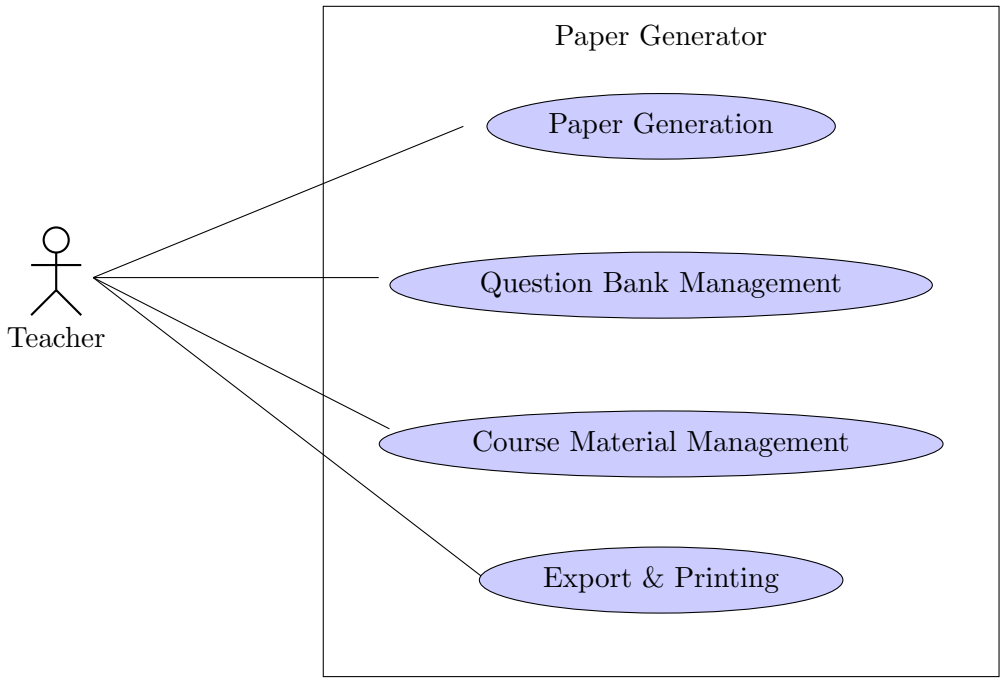
5.1 Use Case Diagram

The following use case diagram illustrates the main interactions between the actors (Teacher and Admin) and the Paper Generator Web Application:



5.2 Use Case Diagram: Paper Generation

The following diagram shows the detailed use case for the Paper Generation feature:



Use Case Description: Paper Generation

Actors: Teacher (Primary Actor)

Goal: To generate quizzes, assignments, midterms, and final examination papers by using questions from the Question Bank or uploaded Course Materials.

Preconditions:

- The Teacher is authenticated and logged into the system.
- Relevant questions exist in the Question Bank and/or Course Materials have been uploaded.

Postconditions:

- A draft paper is generated based on the Teacher's selected parameters (exam type, difficulty, number of questions).
- The Teacher may choose to export the finalized paper for distribution.

Main Flow:

1. The Teacher selects the *Paper Generation* option.
2. The system prompts the Teacher to provide exam details such as type (quiz, assignment, midterm, final), subject, and difficulty level.
3. The system retrieves questions from the **Question Bank** and/or processes uploaded **Course Material**.
4. The system compiles the exam paper according to the selected parameters.
5. The Teacher reviews the draft paper and finalizes it.
6. Optionally, the Teacher uses the *Export & Printing* feature to save or distribute the paper in PDF/Word format.

Alternative Flows:

- If there are insufficient questions in the Question Bank or Course Material, the system alerts the Teacher to add or upload more questions.
- If the Teacher cancels the process, no paper is generated.

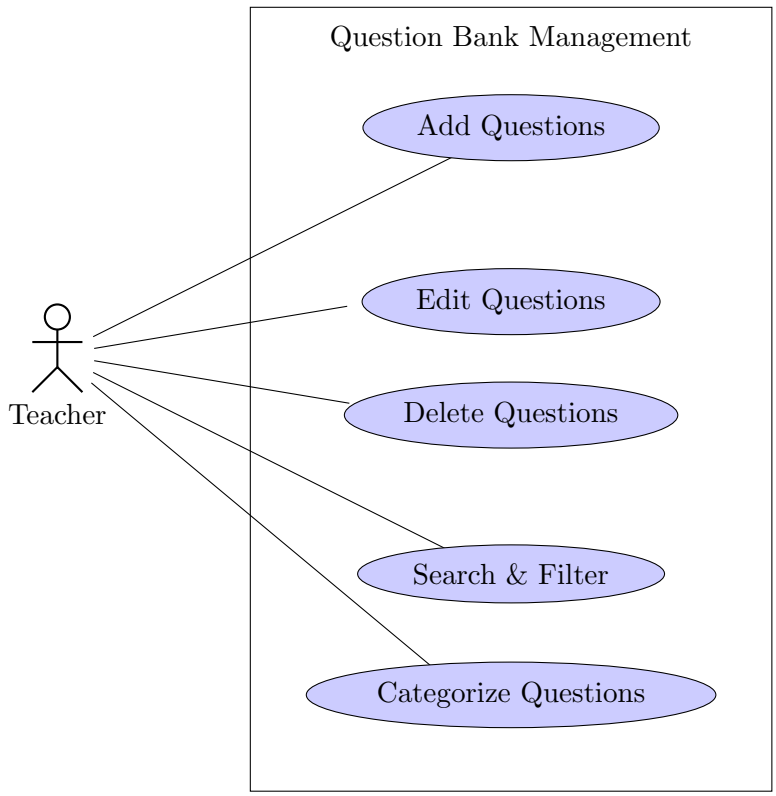
Extensions:

- **Export & Printing** is modeled as an `<<extend>>` use case, providing optional functionality to produce distribution-ready documents.

Remarks: This use case is the central functionality of the Paper Generator Web Application. It includes both *Question Bank Management* and *Course Material Management* through `<<include>>` relationships, ensuring that paper generation is always based on valid and organized content.

5.3 Use Case Diagram: Question Bank Management

The following diagram shows the detailed use case for managing the Question Bank:



Use Case Description: Question Bank Management

Actors: Teacher (Primary Actor)

Goal: To maintain a centralized repository of questions by adding, editing, deleting, categorizing, and retrieving them efficiently.

Preconditions:

- The Teacher is logged into the system.
- The Teacher has the necessary permissions to manage questions.

Postconditions:

- The Question Bank is updated according to the Teacher's actions.
- All questions are stored with relevant metadata such as subject, topic, type, and difficulty level.

Main Flow:

1. The Teacher accesses the *Question Bank Management* feature from the system.
2. The Teacher may perform the following actions:
   - Add new questions manually or import them from uploaded material.
   - Edit existing questions to correct or update their content.
   - Delete questions that are outdated or no longer valid.
   - Categorize questions by subject, topic, type (MCQ, short, long), and difficulty (Easy, Moderate, Hard).
   - Search and filter questions using keywords, categories, or difficulty level.
3. The system validates all changes and updates the Question Bank accordingly.

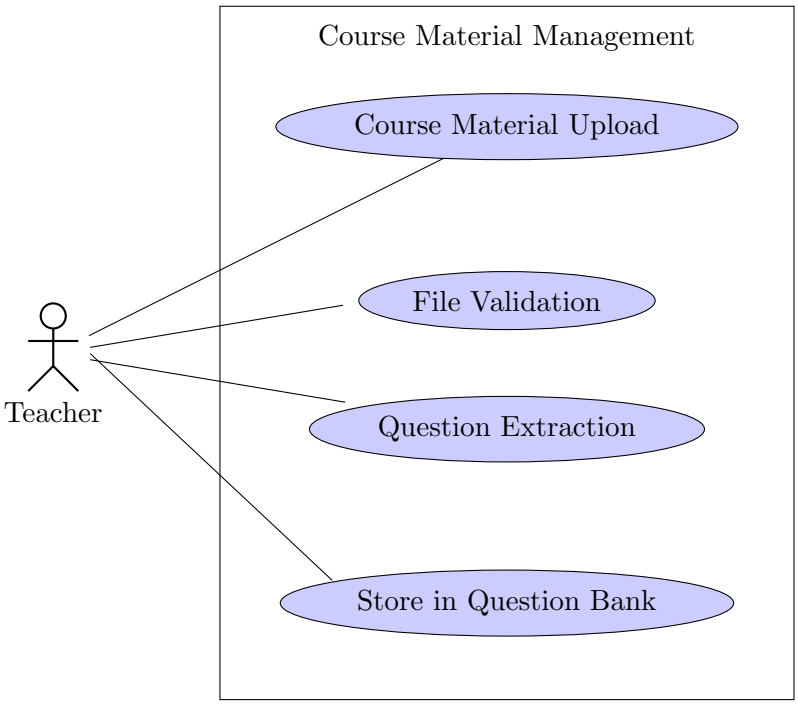
Alternative Flows:

- If a duplicate question is detected, the system warns the Teacher and prevents duplication.
- If the Teacher attempts to delete a question linked to an existing paper, the system requests confirmation before removal.

Remarks: This use case ensures that the Question Bank remains accurate, organized, and scalable. It plays a crucial role in supporting the *Paper Generation* process by providing a reliable set of categorized and validated questions.

5.4 Use Case Diagram: Course Material Management

The following diagram shows the detailed use case for the Course Material Management feature:



Use Case Description: Course Material Management

Actor: Teacher (Primary Actor)

Goal: To upload course materials (PDFs, Word documents, or textbooks), validate them, extract questions, and store them in the Question Bank for later use.

Preconditions:

- The Teacher is logged into the system.
- The Teacher has access to digital course material in supported formats (PDF, Word).

Postconditions:

- Course material is successfully uploaded and validated.
- Extracted questions are stored in the Question Bank with appropriate metadata.

Main Flow:

1. The Teacher selects the option to upload course material.
2. The system performs *File Validation* (format and file size checks).
3. If valid, the system extracts questions from the material.
4. Extracted questions are automatically categorized and stored in the Question Bank.

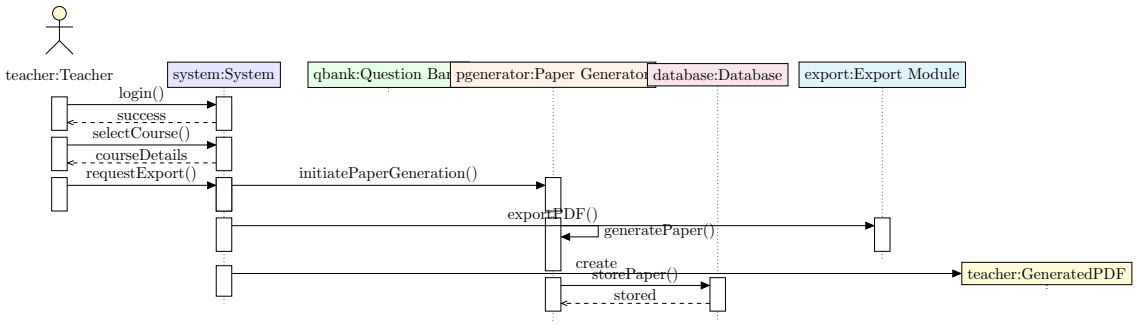
Alternative Flows:

- If the uploaded file is in an unsupported format, the system shows an error message.
- If extraction fails due to poor formatting, the system notifies the Teacher and logs the issue.

Remarks: This use case ensures that teachers can directly integrate their course materials into the system. By extracting and storing questions automatically, it reduces manual workload and keeps the Question Bank updated with relevant academic content.

5.5 Sequence Diagram: Paper Generation

The following refined sequence diagram illustrates the interaction between the Teacher and the Paper Generator Web Application during the paper creation and export process.



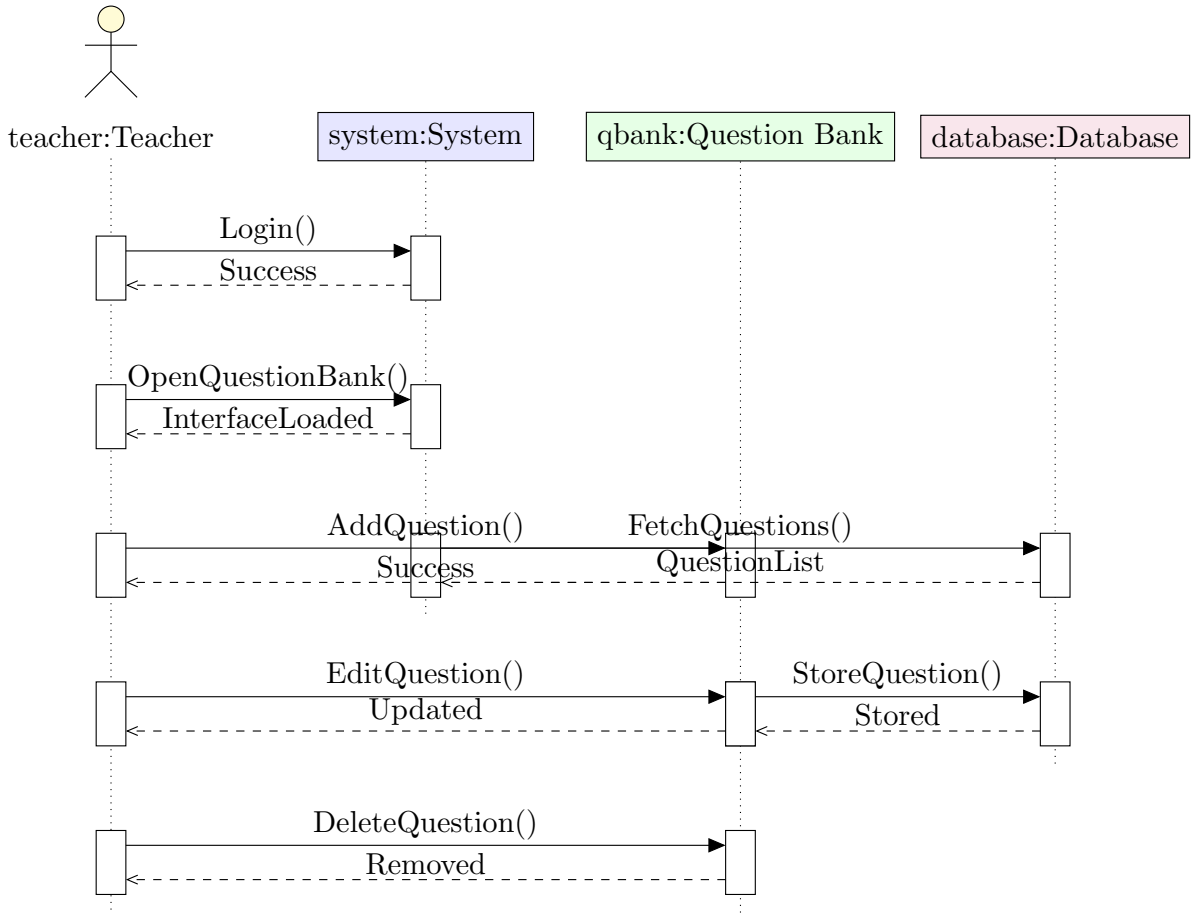
Description

This sequence diagram illustrates the paper generation workflow:

- **Authentication**: Teacher logs into the System
- **Initialization**: Teacher selects course and initiates paper generation
- **Paper Creation**: Paper Generator composes the exam paper and stores it in the Database
- **Export**: Teacher requests and receives the final paper as PDF via the Export Module

5.6 Sequence Diagram: Question Bank Management

The following sequence diagram illustrates how a Teacher interacts with the system to manage the Question Bank — including adding, editing, and retrieving questions.



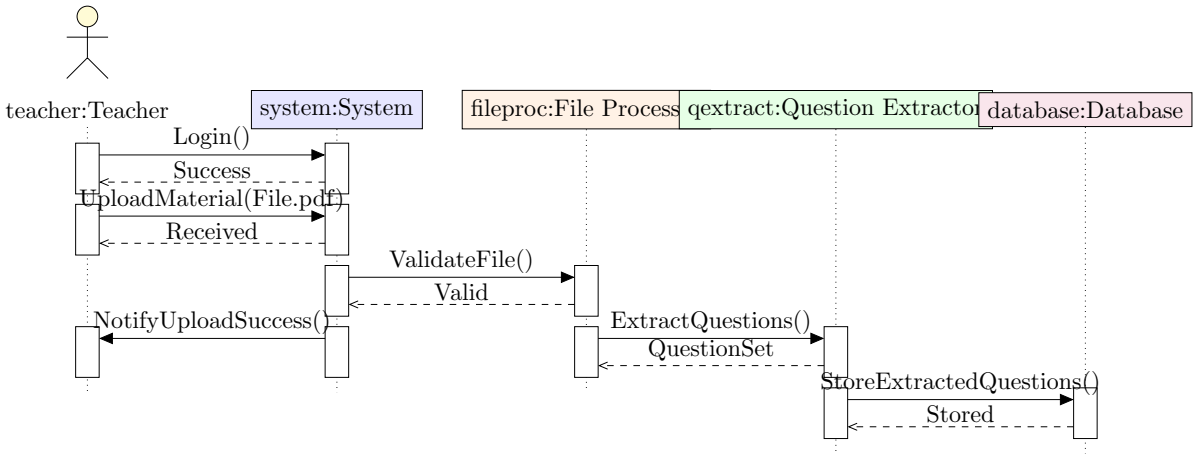
Description

This sequence diagram illustrates the question bank management workflow:

- **Authentication**: Teacher logs into the System
- **Access**: Teacher opens the Question Bank interface
- **Retrieval**: System fetches existing questions from Database
- **Management**: Teacher performs Add, Edit, and Delete operations on questions
- **Persistence**: All changes are stored in the Database

5.7 Sequence Diagram: Course Material Management

The following sequence diagram illustrates how a teacher uploads and manages course materials within the Paper Generator Web Application.

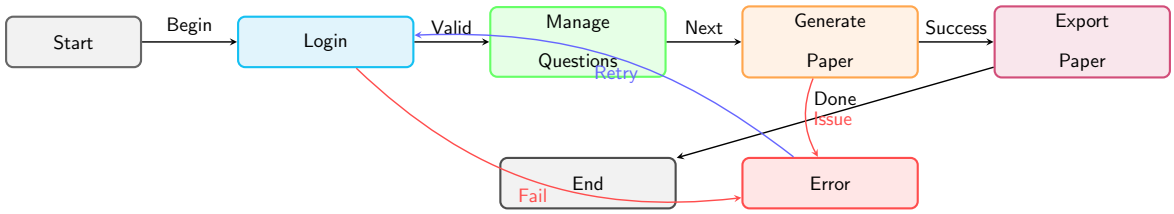


Description: This sequence diagram illustrates the course material upload workflow:

- **Authentication**: Teacher logs into the System
- **Upload**: Teacher submits course material file
- **Processing**: System validates file and extracts questions
- **Storage**: Extracted questions are saved to Database
- **Confirmation**: Teacher receives upload success notification

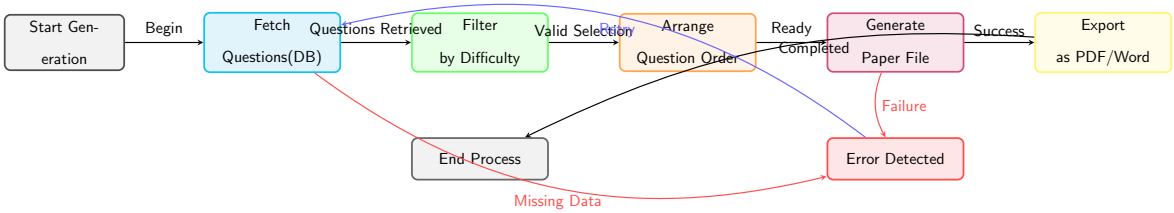
5.8 System State Diagram: Paper Generator Application

The following state diagram presents the main flow of the Paper Generator Web Application.



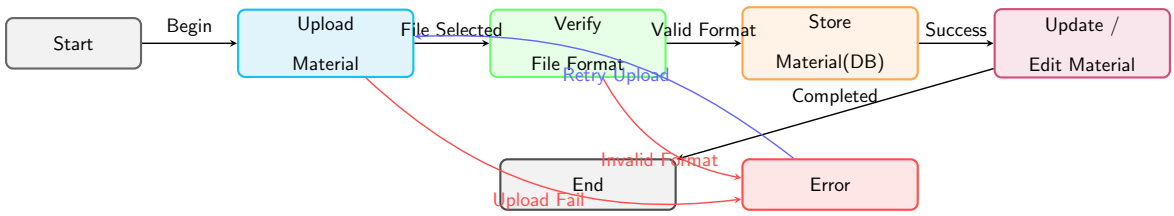
5.9 State Diagram: Paper Generation Process

The following state diagram illustrates the workflow of the Paper Generation feature, showing the transitions between different system states.



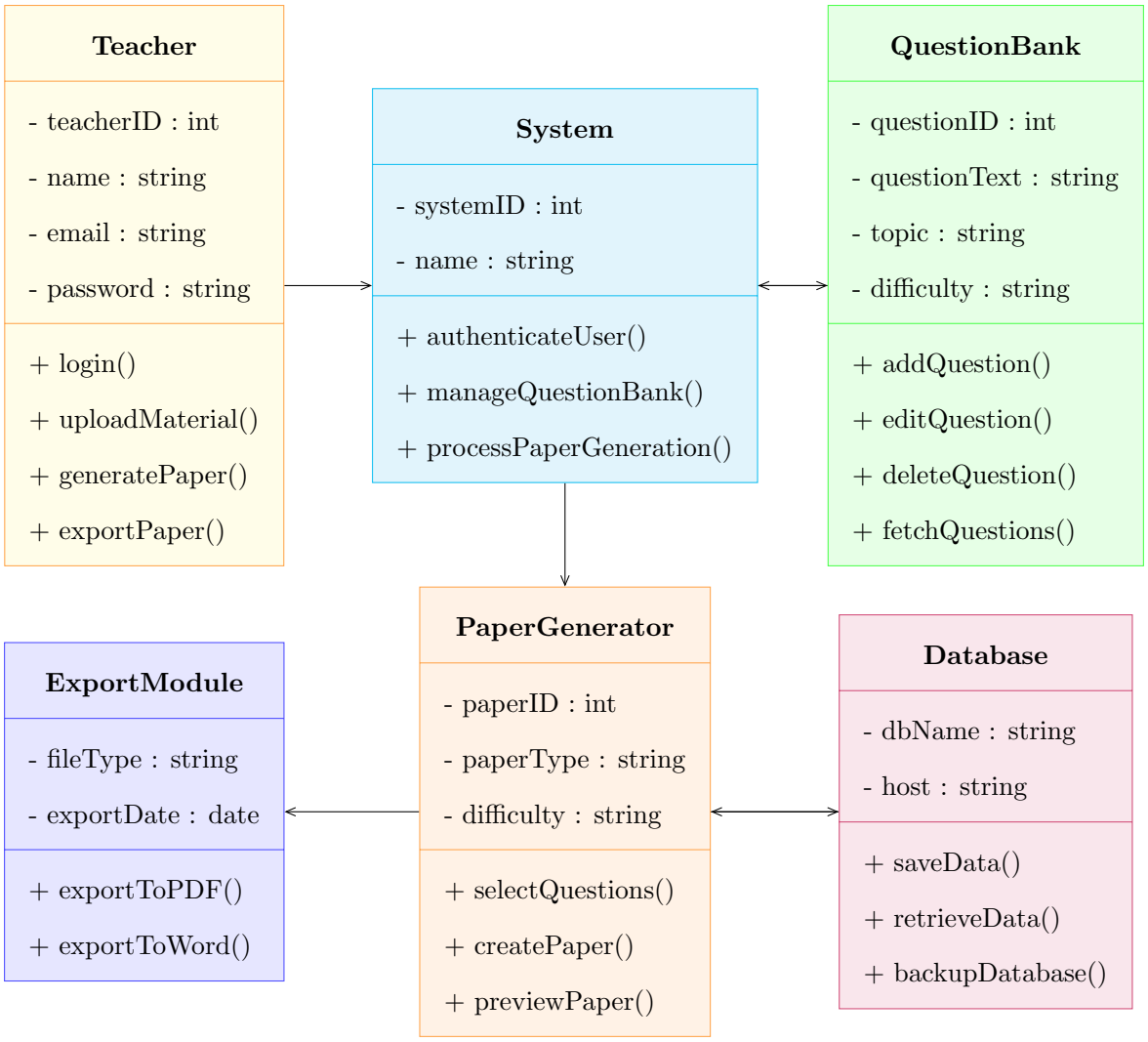
5.10 State Diagram: Course Material Management

The following state diagram illustrates the lifecycle of course material management within the Paper Generator Web Application, showing how teachers upload, verify, and manage their teaching resources.



5.11 Class Diagram: Paper Generator Web Application

The following class diagram represents the structure of the main classes in the system, their attributes, operations, and relationships.



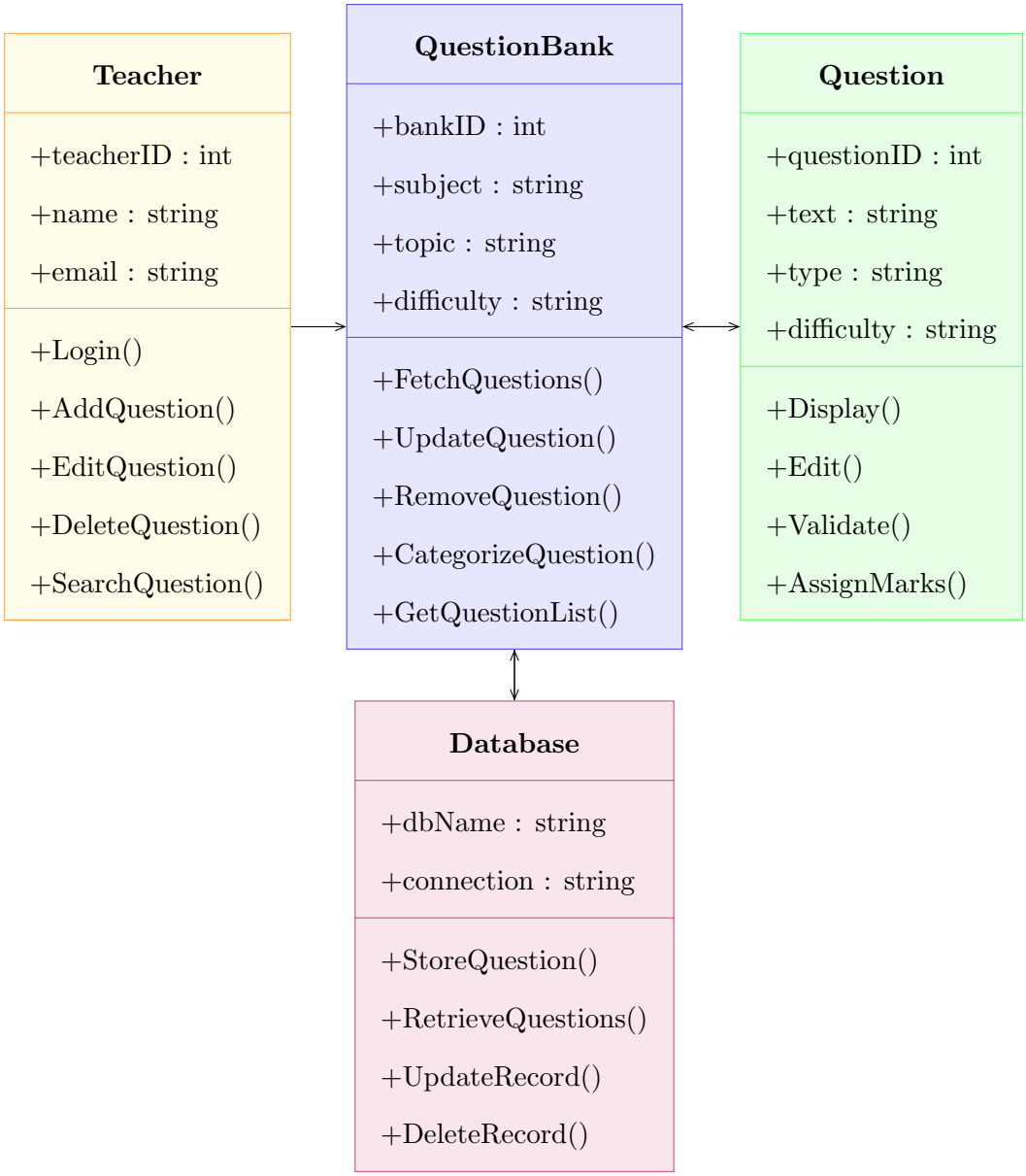
Description:

The Class Diagram above represents the main structural components of the Paper Generator Web Application. The **Teacher** class interacts with the **System** to authenticate users and initiate operations such as paper generation and exporting. The **System** manages the workflow by communicating with the **QuestionBank** to retrieve questions and coordinating with the **PaperGenerator** for assembling examination papers. The **PaperGenerator** further interacts with the **Database** to store and retrieve data, while the **ExportModule** handles the final conversion of generated papers into printable formats such as PDF or Word. The relationships shown in the diagram highlight a modular,

scalable, and maintainable system design, aligning with standard object-oriented development principles.

5.12 Class Diagram: Question Bank Management

The following diagram represents the structure and interaction of components involved in managing the question bank within the system.



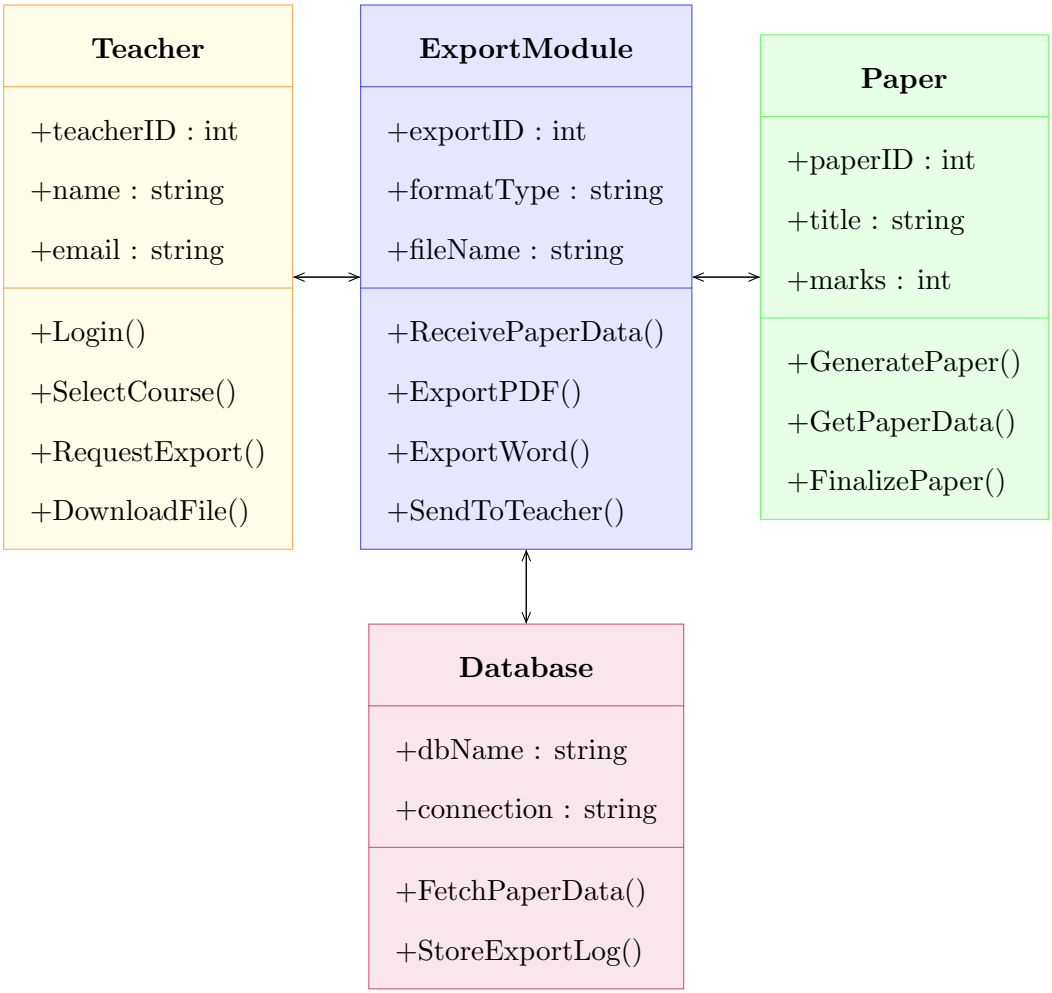
Description:

This class diagram illustrates the interactions within the **Question Bank Management** module. The **Teacher** manages questions through the system by adding, editing, deleting, and searching them. These operations are handled by the **QuestionBank** class,

which organizes questions by subject, topic, and difficulty. Each individual **Question** object contains its specific content and attributes, such as text, type, and marks. The **Database** class ensures reliable data storage and retrieval, keeping the question records synchronized. This diagram aligns with the behavior defined in the sequence diagram for consistent representation of the system's logic.

5.13 Class Diagram: Export Module

The following diagram illustrates the internal structure of the Export Module, aligned with the operations shown in the sequence diagram.

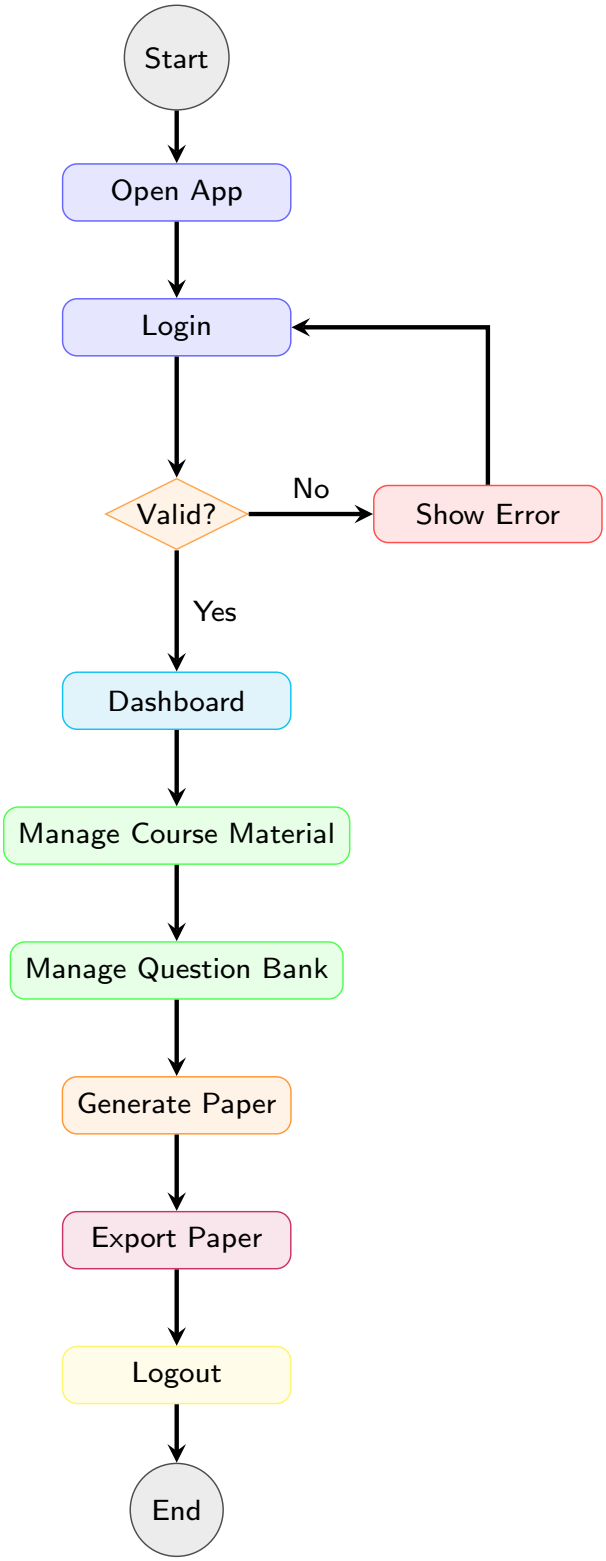


Description:

This class diagram represents the structure of the **Export Module** in relation to the entire paper generation workflow. The **Teacher** logs in, selects a course, and requests an export of the generated paper. The **ExportModule** receives paper data, converts it into supported formats (PDF or Word), and sends the exported file to the teacher. The **Paper** class handles generation and preparation of paper content, while the **Database** ensures data retrieval and maintains export history. This consistent interaction flow ensures system reliability and aligns with the behavior illustrated in the sequence diagram.

5.14 System Activity Diagram: Paper Generator Application

The following activity diagram illustrates the main operational flow of the Paper Generator Web Application.

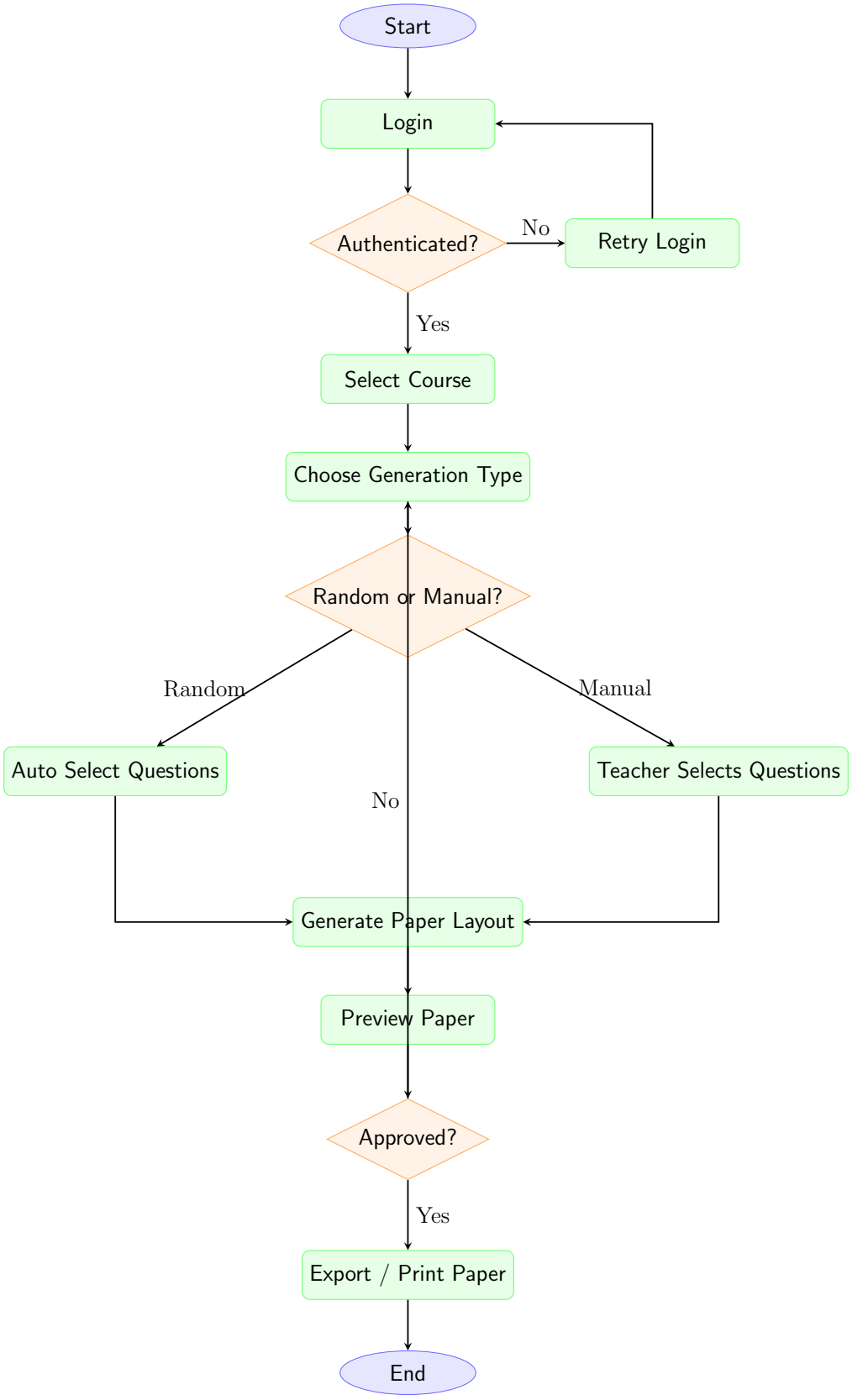


Description:

This activity diagram illustrates the primary workflow of the Paper Generator Web Application. The process begins when the teacher accesses the system and logs in successfully. After authentication, the user may upload course materials, manage the question bank, and initiate the paper generation process. Once the paper is created, it can be exported for printing or digital distribution. The activity flow ends when the user logs out of the system. In case of failed authentication, the user is redirected back to the login stage. This diagram represents the overall operational behavior of the system in a clear and structured manner.

5.15 Activity Diagram: Paper Generation

The following activity diagram illustrates the workflow followed by a teacher to generate an exam paper within the system.



Description:

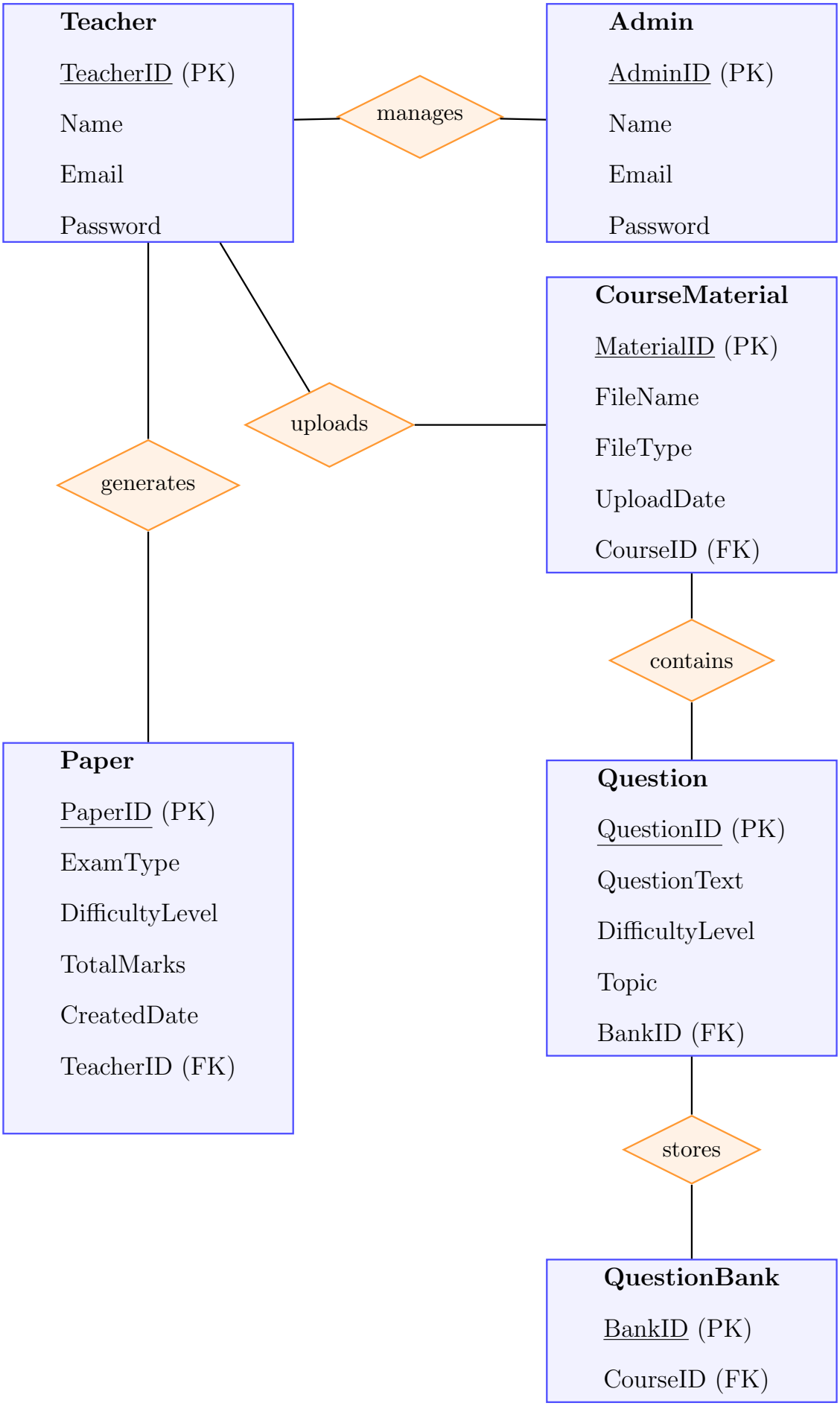
The above activity diagram illustrates the complete workflow followed by a teacher to generate an examination paper using the Paper Generator Web Application. The process begins when the teacher logs into the system. After successful authentication, the teacher selects the relevant course and chooses whether the paper will be generated manually or randomly.

If random generation is selected, the system automatically retrieves suitable questions from the question bank based on predefined criteria. In manual mode, the teacher individually selects questions. Once selected, the system arranges the questions into a structured exam layout, which is then presented to the teacher for preview.

If the teacher approves the paper, it can be exported or printed for academic use. If not approved, the teacher may revise the selection or regenerate the paper. In case of failed authentication, the system prompts the teacher to retry login. The process concludes once the paper has been successfully exported or printed.

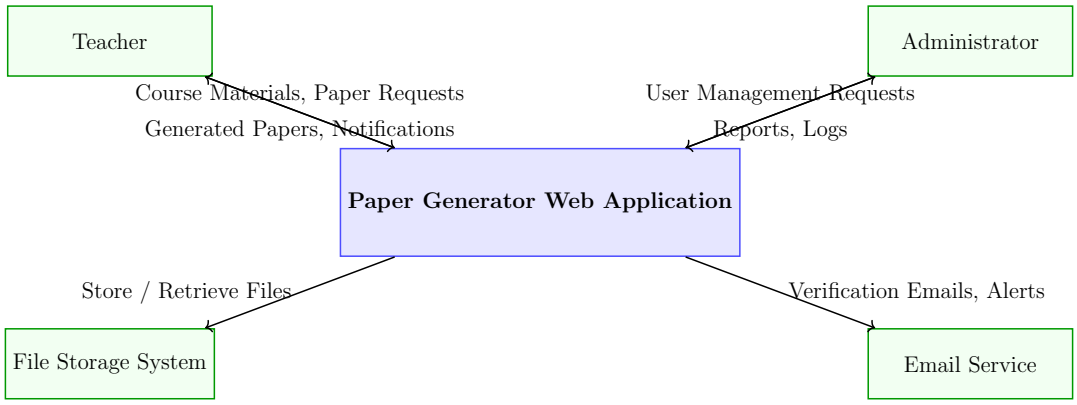
5.16 ER Diagram

The ER diagram illustrates the system entities along with their attributes, primary keys, and foreign key relationships for the Paper Generator Web Application.


Teacher
TeacherID (PK)
Name
Email
Password
manages
Admin
AdminID (PK)
Name
Email
Password
CourseMaterial
MaterialID (PK)
FileName
FileType
UploadDate
CourseID (FK)
uploads
generates
contains
Paper
PaperID (PK)
ExamType
DifficultyLevel
TotalMarks
CreatedDate
TeacherID (FK)
Question
QuestionID (PK)
QuestionText
DifficultyLevel
Topic
BankID (FK)
stores
QuestionBank
BankID (PK)
CourseID (FK)

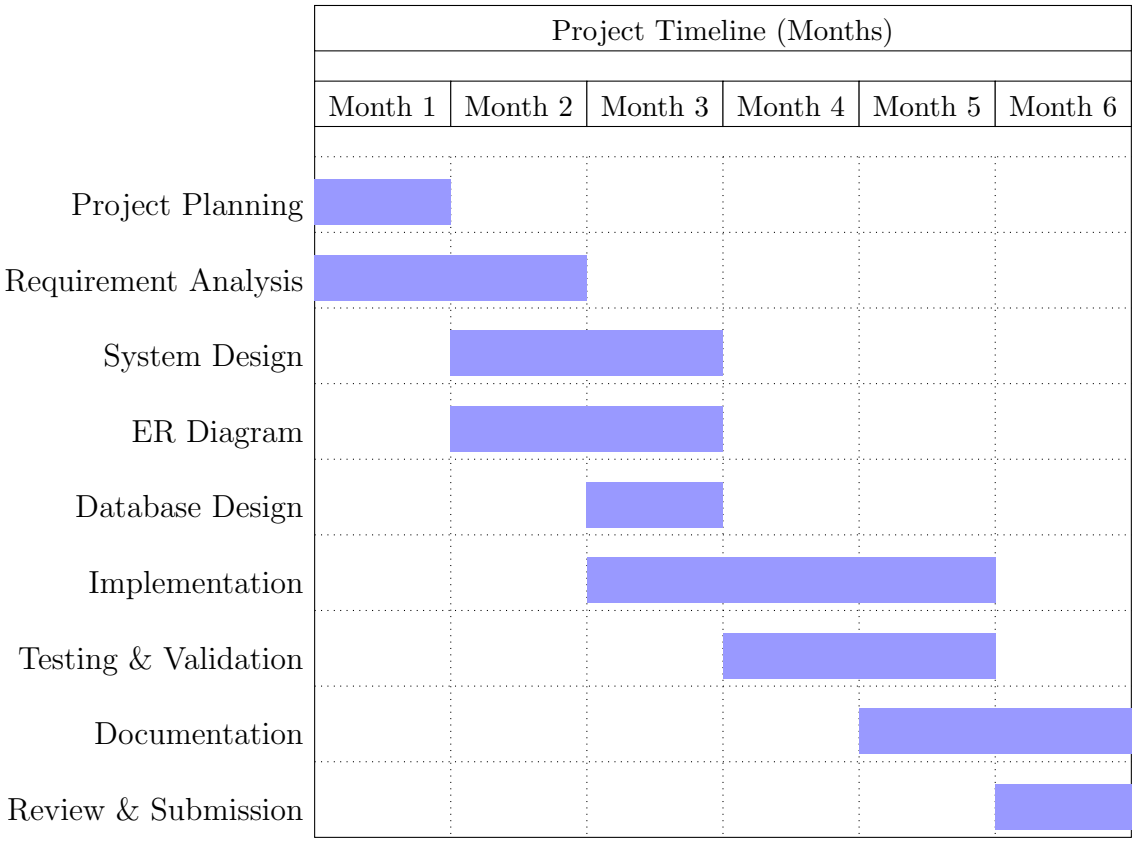
5.17 Context Diagram

The context diagram shows the Paper Generator Web Application as a single system and illustrates its interaction with external entities.



5.18 Gantt Chart

The Gantt chart illustrates the project schedule for the development of the Paper Generator Web Application, showing the timeline and sequence of major activities.



6 Test case:

6.1 Test Case 1: Generate Paper with Valid Inputs

Test Case Template			
Project Name	Paper Generator Web Application	**Test Designed By**	Talha Niaz
Test Case ID	PG-TC-01	**Test Design Date**	11-01-2026
Test Priority	High	**Test Executed By**	—
Module Name	Paper Generation	**Test Execution Date**	—
Test Title: Generate paper with valid inputs			
Description: Verify that the system generates an exam paper successfully when valid exam parameters are provided by the teacher.			
Preconditions: Teacher is logged in and sufficient questions exist in the Question Bank.			
Dependencies: Question Bank Management, Course Material Management			
Test Steps			
Step	**Test Steps**	**Test Data**	**Expected Result**
1	Select Paper Generation option	—	Paper generation interface is displayed
2	Enter exam type	Midterm	System accepts exam type
3	Select subject	Database Systems	Subject is selected successfully
4	Choose difficulty level	Moderate	Difficulty level is accepted
5	Click Generate Paper	—	Draft paper is generated successfully
Actual Result: Draft paper generated successfully			
Status: Pass			
Notes: —			
Postconditions: Generated paper is available for review and export.			

6.2 Test Case 2: Generate Paper with Insufficient Questions

Test Case Template			
Project Name	Paper Generator Web Application	**Test Designed By**	Talha Niaz
Test Case ID	PG-TC-02	**Test Design Date**	11-01-2026
Test Priority	High	**Test Executed By**	—
Module Name	Paper Generation	**Test Execution Date**	—
Test Title: Generate paper with insufficient questions			
Description: Verify that the system displays an alert when there are not enough questions available to generate the selected exam paper.			
Preconditions: Teacher is logged in and the Question Bank contains insufficient questions for the selected parameters.			
Dependencies: Question Bank Management, Course Material Management			
Test Steps			
Step	**Test Steps**	**Test Data**	**Expected Result**
1	Select Paper Generation option	—	Paper generation interface is displayed
2	Enter exam details	Final Exam, Hard	System accepts input parameters
3	Click Generate Paper	—	System checks question availability
4	Insufficient questions detected	—	System displays warning message
Actual Result: System displays alert indicating insufficient questions.			
Status: Pass			
Notes: Teacher is prompted to add more questions or upload course material.			
Postconditions: No paper is generated until sufficient questions are available.			

6.3 Test Case 3: Manage Question Bank

Test Case Template			
Project Name	Paper Generator Web Application	**Test Designed By**	Talha Niaz
Test Case ID	PG-TC-03	**Test Design Date**	12-01-2026
Test Priority	High	**Test Executed By**	—
Module Name	Question Bank Management	**Test Execution Date**	—
Test Title: Manage Question Bank operations			
Description: Verify that the system allows the Teacher to add, edit, delete, categorize, and retrieve questions successfully in the Question Bank.			
Preconditions: Teacher is logged in and has permission to manage the Question Bank.			
Dependencies: User Authentication, Course Material Management			
Test Steps			
Step	**Test Steps**	**Test Data**	**Expected Result**
1	Open Question Bank Management module	—	Question Bank interface is displayed
2	Add a new question	MCQ, Easy, Software Engineering	Question is saved successfully
3	Edit existing question	Update question text	Question is updated in the system
4	Search and filter questions	Keyword, Difficulty	Relevant questions are displayed
5	Delete a question	Selected question	System removes the question after confirmation
Actual Result: System successfully manages Question Bank operations.			
Status: Pass			
Notes: System prevents duplication and confirms deletion of linked questions.			
Postconditions: Question Bank is updated with accurate and categorized questions.			